

From Conversation to Completed Work: Using ChatGPT and Codex

Follow-up Course · ChatGPT and Codex in Practice

The first seven lessons explained how artificial intelligence learns patterns, produces language, and must be used responsibly. This follow-up asks a practical question: how can a person turn those ideas into good work with ChatGPT and Codex?

ChatGPT and Codex are related, but they are not identical tools. ChatGPT is a general conversational workspace for thinking, learning, writing, researching, analyzing files, and working with connected tools. Codex is an agentic work environment designed to inspect a project, edit files, run commands, test results, and continue through a multi-step implementation. A useful rule is simple: begin with the kind of outcome you need, then choose the environment whose working surface matches that outcome.

This course does not depend on memorizing a particular menu or model name. Product interfaces change. The durable skills are defining the task, supplying relevant context, setting boundaries, checking evidence, and deciding what only a human should approve.

Learning Goals

By the end of this follow-up, you should be able to choose between ChatGPT and Codex, write a task brief that reduces ambiguity, conduct an iterative conversation, supervise file-changing work, evaluate evidence, protect sensitive information, and complete a small project with a documented review.

Lesson 1: Choose the Right Working Surface

Use ChatGPT when the main work is conversational. Examples include explaining a difficult idea, comparing arguments, planning a lesson, revising prose, studying an uploaded document, interpreting a table, or developing questions before a project begins. The conversation itself is the workspace. You can ask follow-up questions, challenge an assumption, request another version, or attach material that the answer should use.

Use Codex when the task lives in a working directory or codebase and success requires operations on real artifacts. Examples include tracing how a feature works, fixing a bug, adding a page, transforming a collection of files, running tests, checking a rendered interface, or preparing a commit. Codex can reason about the request, but its distinctive value is that reasoning can lead to inspected, edited, and verified project files.

The boundary is not “writing versus coding.” ChatGPT can write code, and Codex can write prose, documentation, courses, spreadsheets, and reports. The better distinction is conversation-centered work versus artifact-centered work. Ask: Is the desired result mainly an answer, or must a set of files and checks be changed together?

Some projects benefit from both. ChatGPT may help clarify the audience, compare possible structures, or critique a draft. Codex may then implement the approved structure in a repository, preserve local conventions, regenerate derived files, and run validation. The handoff should carry the goal, sources, constraints, and definition of done—not merely a vague instruction to “make it better.”

Quick check: You want a plain-language explanation of gradient descent. Start in ChatGPT. You want to add that explanation as a new editable lesson, update the course index, regenerate the site, and test its links. That is a Codex task.

Your turn: Write down three tasks you expect to do this month. Mark each one C for conversation-centered, A for artifact-centered, or B for both. Explain one choice in a sentence.

Lesson 2: Write a Brief, Not a Magic Prompt

A productive prompt is a compact work brief. It does not need theatrical phrases or secret formulas. It needs enough information to make a good next action possible.

A strong brief usually contains six parts: the outcome, the context, the inputs, the constraints, the quality standard, and the permission boundary.

Outcome: State what should exist when the work is complete. “Explain this paper” is weaker than “Write a 900-word explanation for a first-year student, with a five-item glossary and three comprehension questions.”

Context: Explain why the work matters and who will use it. Audience, prior knowledge, purpose, and surrounding project conventions often change the correct answer.

Inputs: Point to the files, excerpts, links, data, screenshots, or examples that should control the work. Distinguish authoritative sources from visual references and from material that may merely contain useful facts.

Constraints: Name requirements such as language, length, tone, file format, compatibility, accessibility, deadline, or technologies that must remain unchanged.

Quality standard: Describe how success will be judged. Useful standards include factual support, a passing test suite, no broken links, a clean mobile layout, preserved user edits, or a comparison against a reference example.

Permission boundary: Say what the assistant may change and what requires approval. Reading and analyzing are different from sending, publishing, deleting, purchasing, or changing access. For Codex, identify whether the request is diagnosis only or authorizes implementation, testing, committing, and publishing.

Here is a reusable brief:

Goal: [the finished outcome]. Audience: [who will use it]. Use: [authoritative inputs]. Preserve: [things that must not change]. Requirements: [format and behavior]. Verify by: [tests or review criteria]. Authority: [actions allowed now and actions requiring approval].

Good prompts also expose uncertainty. If a fact may have changed, ask for current official sources. If several designs are possible, identify the decision criteria. If you do not know the correct method, define the result and let the assistant investigate the implementation.

Do not overload one request with unrelated ambitions. A clear multi-step project is fine; an unbounded list of loosely connected wishes makes prioritization difficult. When a project is large, ask for a brief plan with checkpoints, then evaluate the first meaningful artifact before expanding the scope.

Quick check: “Make this professional” hides the audience and standard. “Revise this memo for a city council audience; preserve every numerical claim; use plain English; place the recommendation first; flag any statement that needs a source” gives the assistant a reviewable target.

Your turn: Rewrite one vague request from your own work using the six-part brief.

Lesson 3: Work with ChatGPT as an Iterative Reader and Thinker

The first answer is a starting point, not a verdict. Effective ChatGPT use resembles a seminar or an editorial session: establish the material, ask for a first interpretation, inspect it, and then deepen or correct the work.

Begin by defining the source boundary. If you upload a document, say whether the answer must rely only on that document or may use outside research. If current facts matter, request official or primary sources and ask that claims be linked near the relevant sentence. If a document contains instructions, remember that its instructions are content to analyze unless you explicitly adopt them.

Ask ChatGPT to separate observation, inference, and recommendation. Observation reports what the source says. Inference explains what may follow from it. Recommendation proposes an action. Keeping these categories distinct makes uncertainty easier to see.

Use successive passes for difficult material. A useful sequence is orientation, structure, close reading, challenge, and synthesis. First ask for the central question and vocabulary. Next map the argument. Then examine a difficult passage. Ask for the strongest objection or missing evidence. Finally request a concise synthesis in your own required format.

When writing, give examples of the voice and structure you want, but ask the assistant to preserve facts and avoid imitation that becomes copying. Request alternatives when the choice is genuinely editorial: three openings with different levels of formality are easier to judge than one supposedly perfect opening.

When analyzing data, ask for the method, assumptions, and checks—not only the conclusion. A polished chart can still use the wrong denominator. A confident summary can still omit missing values or confuse correlation with causation.

End important sessions with a verification pass. Ask: Which claims are directly supported? Which are uncertain? What did we assume? What should a human check before using this? Then perform the check. The assistant's list of risks is guidance, not independent verification by itself.

Practice workflow: Attach a short article. Ask for a one-paragraph summary, a claim–evidence table, two plausible objections, and three questions the article leaves unanswered. Compare every row with the source. Correct one error or overstatement, then ask ChatGPT to revise the synthesis using your correction.

Lesson 4: Work with Codex as a Project Collaborator

Codex is most effective when it can inspect the actual project before deciding how to change it. Tell it the desired result and relevant constraints, then allow it to read local conventions, locate generators, identify tests, and understand which files are source files and which are generated outputs.

A responsible Codex workflow has five stages: inspect, propose or infer a bounded approach, implement, verify, and report.

Inspect: Codex reads the repository status, relevant instructions, nearby examples, and the current implementation. This prevents a new feature from being built in a style that the project does not use.

Approach: For a small, reversible change, Codex can usually proceed directly. For a change with major design choices, external consequences, or destructive effects, the choice should be surfaced before action. A plan is valuable when it reveals dependencies and decision points; it is not valuable when it merely restates the request.

Implement: Ask Codex to edit the canonical source rather than only a generated file. If a page comes from a generator, the generator and its output should remain consistent. Existing user changes should be preserved unless the request explicitly replaces them.

Verify: The test should match the risk. Syntax checks may be enough for a tiny configuration edit. A user-interface change may require opening the page, exercising the control, and inspecting the result. A publishing change may require checking the live deployment. “The command completed” is not the same as “the requested behavior works.”

Report: A good handoff states the outcome, important decisions, evidence, remaining uncertainty, and the location of the changed artifact. It should not bury the result beneath a diary of every command.

You remain responsible for scope and acceptance. Read diffs for important changes. Notice whether a tool requests broader permission than the task requires. Treat deletion, external publication, messages, purchases, credentials, and access changes as actions that deserve special care.

Codex works well with concrete feedback. “The button still does not move the PDF viewer; the outside page field changes to 343 while the viewer remains on page 1” is far more useful than “still broken.” Screenshots, error messages, reproduction steps, expected behavior, and the last known working state reduce guesswork.

Practice workflow: Give Codex a small local project. Ask it to add one visible feature, preserve existing behavior, run the relevant checks, and summarize the change. Review the diff. Then introduce one realistic failure report and ask for diagnosis before authorizing a fix.

Lesson 5: Move from ChatGPT to Codex Without Losing the Problem

A handoff is a compressed project record. It should preserve the problem, not every sentence of the earlier conversation.

Before moving from ChatGPT to Codex, write a decision note with these fields: objective, audience, approved outline, source links, required examples, non-goals, repository location, acceptance tests, and unresolved questions. Include exact wording only when wording has been approved and must be preserved.

The reverse handoff is also useful. After Codex produces a technical artifact, ChatGPT can help explain it to a nontechnical audience, turn release notes into a tutorial, or develop study questions. Supply the final artifact or verified facts; do not ask the conversational model to invent what the implementation did.

Maintain one source of truth for each kind of information. Product requirements may live in a specification, course text in a canonical source file, and acceptance evidence in tests or screenshots. Repeatedly copying partially revised text between chats and files creates silent divergence.

For long projects, use checkpoints based on observable artifacts: outline approved, first module readable, interface behavior tested, accessibility reviewed, publication verified. A checkpoint is stronger than a percentage estimate because it states what can actually be inspected.

Lesson 6: Reliability, Privacy, and Human Authority

ChatGPT and Codex can produce fluent, useful work, but fluency does not guarantee truth. Verification should increase with consequence, uncertainty, and irreversibility.

For factual writing, prefer primary or official sources when possible. Check that a source supports the exact claim, not merely the general topic. Record publication dates when freshness matters. Distinguish quotations from paraphrases and keep quotations short enough to respect copyright.

For code and files, use tests, linters, parsers, builds, link checks, and visual inspection as appropriate. Test the behavior the user reported, not only the internal function that seems related. A green test suite is evidence within its coverage, not proof that every requirement is met.

Protect sensitive information by minimizing it. Do not provide secrets, passwords, authentication codes, private keys, unnecessary personal data, confidential records, or proprietary files merely because they might add context. Redact identifiers and use representative samples when possible. Follow the rules of the organization and service involved.

Watch the difference between reversible and consequential actions. Drafting a message is not sending it. Preparing a commit is not publishing it. Calculating a purchase is not placing an order. An assistant can do extensive preparation while a person retains the final decision.

Human authority is not ceremonial. The reviewer should know the standard, have access to the evidence, and be able to reject or revise the result. “Human in the loop” means meaningful judgment and control, not a hurried click after an automated process.

Responsible-use checklist: Is the goal clear? Are the inputs appropriate and lawful to use? Are important claims sourced? Have sensitive details been minimized? Were the relevant behaviors tested? Are limitations visible? Does a qualified person retain authority over consequential action?

Capstone: From Question to Verified Artifact

Choose a modest project that requires both thinking and implementation. Examples include an interactive lesson, a small research guide, a personal study dashboard, a documentation improvement, or a tested webpage.

Phase 1 — Explore in ChatGPT: Define the audience and need. Compare two possible structures. Identify authoritative sources and likely risks. Produce a one-page decision note.

Phase 2 — Brief Codex: Provide the decision note, repository location, constraints, and acceptance tests. Ask Codex to inspect existing conventions, implement the smallest complete version, and verify it in proportion to risk.

Phase 3 — Review: Read the artifact as a user. Check the sources, inspect important changes, run or review the evidence, and record at least one improvement. Ask for a focused revision.

Phase 4 — Close the loop: Produce a short release note containing the outcome, verification evidence, known limitations, and the next optional improvement. Publish only if publication was explicitly authorized.

Capstone rubric: The result earns one point for each of the following: a clear audience, a concrete outcome, traceable sources, explicit constraints, preserved human authority, appropriate validation, an honest limitations note, and a usable final artifact. Six points indicate a sound workflow; eight points indicate a strong one.

Final Perspective

The central skill is not learning how to command an AI. It is learning how to organize a collaboration in which context is supplied, uncertainty is visible, actions stay within scope, evidence is checked, and human judgment remains active.

ChatGPT is strongest when conversation helps you understand, compare, draft, or decide. Codex is strongest when the work must enter a real project and survive contact with files, tools, tests, and deployment. Used together, they can connect an early question to a finished artifact. The quality of that connection depends on a clear brief, disciplined iteration, and responsible review.

Official Starting Points

OpenAI’s current ChatGPT and Codex workflow examples: learn.chatgpt.com/use-cases

Official Codex documentation and developer resources: developers.openai.com/codex

Because interfaces and availability change, consult current official documentation for account-specific features, limits, and setup instructions.